Question 1

Practical Assignment: Blog Management REST API with Authentication (Node.js, Express, MongoDB, EJS)

1. User Authentication

● POST /api/auth/register – Register a new user (name, email, password) with the password hashed using bcrypt.

● POST /api/auth/login – Authenticate a user and return a JWT token.

● GET /api/auth/profile – A protected route that returns the logged-in user's profile.

2. Blog Post CRUD API (protected routes, linked to the logged-in author)

● POST /api/posts – Create a new blog post (title, content, tags, published status).

● GET /api/posts – Retrieve all blog posts belonging to the logged-in user.

● GET /api/posts/:id – Retrieve a single blog post by ID.

● PUT /api/posts/:id – Update a blog post by ID (only the author or an admin can update).

● DELETE /api/posts/:id – Delete a blog post by ID (only the author or an admin can delete).

● POST /api/posts/:id/image – Upload or replace a featured image for a blog post (multipart/form-data) using Multer.

3. EJS Views

● Render an EJS page (views/posts.ejs) that displays all published blog posts in a styled list.

● Render an EJS page (views/login.ejs) with a login form that submits to your login route.

● Render an EJS page (views/dashboard.ejs, protected) showing the logged-in user's own posts with edit/delete links, visible only after login.

4. File Uploading

● Use Multer to handle multipart/form-data uploads for a blog post’s featured image and/or a user’s profile picture.

● Restrict uploads to image MIME types (jpeg, png, webp) and enforce a maximum file size (e.g., 2MB); reject and return a clear error for anything else.

● Store uploaded files in a local /uploads directory (or a cloud service such as Cloudinary/AWS S3) and save only the resulting file path/URL on the Post or User document.

● Serve uploaded images as static assets and render them in views/posts.ejs and views/dashboard.ejs.

● Delete the associated image file from storage when a post (or its image) is deleted or replaced.

5. Security

● Use Helmet to set secure HTTP headers, and configure CORS to only allow trusted origins.

● Apply rate limiting (e.g., express-rate-limit) on /api/auth/login and /api/auth/register to mitigate brute-force and credential-stuffing attacks.

● Validate and sanitize all request input (e.g., with express-validator or Joi) to guard against NoSQL

injection and malformed payloads.

● Keep secrets (JWT signing key, MongoDB URI, upload credentials) in environment variables via dotenv and never commit them to source control (.env.example only).

● Set a reasonable JWT expiry, sign tokens with a strong secret, and reject/refresh appropriately on expiry.

● Validate uploaded file types on the server (not just by extension) and never trust the client-supplied filename when saving to disk.

6. Requirements

● Use bcrypt to hash and compare passwords — never store plain text passwords.

● Use JWT middleware to protect all post-related API and dashboard routes.

● Use Mongoose schemas to model User and Post, with each post referencing its author via ObjectId and each user having a role field (user/admin).

● Ensure a user can only view/edit/delete their own posts, unless they are an admin.

● Return appropriate HTTP status codes and consistent JSON responses for the API routes.

●

● Never trust client input for authorization decisions — always re-check ownership/role server-side, even if the UI hides edit/delete links.

● Apply the same Security requirements above (Helmet, rate limiting, input validation, env-based secrets) across both the API and file-upload routes.

Deliverable: A working Node.js/Express project with proper folder structure (models, routes,

controllers, middleware, views, uploads), a README.md explaining setup and usage (including a

.env.example listing required environment variables), and example API requests (curl or Postman) demonstrating registration, login, all blog post CRUD operations, and image upload, plus screenshots or a demo of the EJS pages.

Code
model

```
const mongoose = require("mongoose");

const postSchema = new mongoose.Schema(
  {
    title: {
```

```
        type: String,
        required: true,
        trim: true,
        minlength: 3,
        maxlength: 150
    },

    content: {
        type: String,
        required: true,
        trim: true
    },

    tags: {
        type: [String],
        default: []
    },

    published: {
        type: Boolean,
        default: false
    },

    image: {
        type: String,
        default: null
    },

    author: {
        type: mongoose.Schema.Types.ObjectId,
        ref: "User",
        required: true
    }
  },
  {
```

```
    timestamps: true
  }
);

module.exports = mongoose.model("Post", postSchema);
``````
------ models/User.js ------
``````
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true,
      minlength: 2,
      maxlength: 50
    },

    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
      trim: true
    },

    password: {
      type: String,
      required: true
    },
```

```
    role: {
      type: String,
      enum: ["user", "admin"],
      default: "user"
    }
  },
  {
    timestamps: true
  }
);

module.exports = mongoose.model("User", userSchema);
```

## MiddleWare

------ middleware/authMiddleware.js ------

```
const jwt = require("jsonwebtoken");

const protect = (req, res, next) => {
  try {
    let token;

    // 1. Check Authorization header
    if (
      req.headers.authorization &&
      req.headers.authorization.startsWith("Bearer ")
    ) {
      token = req.headers.authorization.split(" ")[1];
    }

    // 2. Check browser cookie
```

```
    if (!token && req.cookies && req.cookies.token) {
      token = req.cookies.token;
    }



    if (!token) {

      return res.status(401).json({
        success: false,
        message: "Authentication required"

      });

    }



    const decoded = jwt.verify(token, process.env.JWT_SECRET);



    req.user = decoded;



    next();

  } catch (error) {

    return res.status(401).json({
      success: false,
      message: "Invalid or expired token"

    });

  }

};



const adminOnly = (req, res, next) => {
  if (req.user.role !== "admin") {
    return res.status(403).json({
      success: false,

      message: "Admin access required"

    });

  }
```

```
  next();
};

module.exports = {
  protect,
  adminOnly
};
```

------ middleware/errorMiddleware.js ------

```
const errorHandler = (err, req, res, next) => {

  console.error(err);

  if (err instanceof SyntaxError) {
    return res.status(400).json({
      success: false,
      message: "Invalid JSON"
    });
  }

  if (err.message?.includes("Only JPEG")) {
    return res.status(400).json({
      success: false,
      message: err.message
```

```
    });

  }



  if (err.code === "LIMIT_FILE_SIZE") {
    return res.status(400).json({
      success: false,
      message: "File size must not exceed 2MB"
    });
  }



  res.status(500).json({
    success: false,
    message: err.message || "Internal server error"
  });
};



module.exports = errorHandler;
``````

------ middleware/uploadMiddleware.js ------

``````
const multer = require("multer");
const path = require("path");
const fs = require("fs");
const crypto = require("crypto");
const { fileTypeFromBuffer } = require("file-type");



const uploadDir = path.join(__dirname, "..", "uploads");



if (!fs.existsSync(uploadDir)) {
  fs.mkdirSync(uploadDir, { recursive: true });
```

```
}


const storage = multer.memoryStorage();


const upload = multer({
  storage,


  limits: {

    fileSize: 2 * 1024 * 1024

  },



  fileFilter: (req, file, cb) => {
    const allowedMimeTypes = [
      "image/jpeg",
      "image/png",
      "image/webp"
    ];



    if (!allowedMimeTypes.includes(file.mimetype)) {
      return cb(
        new Error("Only JPEG, PNG and WEBP images are allowed")

      );

    }



    cb(null, true);

  }

});



const validateAndSaveImage = async (req, res, next) => {
  try {
```

```
if (!req.file) {

  return res.status(400).json({
    success: false,
    message: "Image file is required"

  });

}



const detectedType = await fileTypeFromBuffer(req.file.buffer);



const allowedTypes = {
  jpeg: "jpg",
  png: "png",
  webp: "webp"
};



if (

  !detectedType ||

  !allowedTypes[detectedType.ext]

) {

  return res.status(400).json({
    success: false,
    message: "Invalid image file"

  });

}



const extension = allowedTypes[detectedType.ext];



const filename =

  crypto.randomUUID() + "." + extension;
```

```
    const filePath = path.join(uploadDir, filename);

    fs.writeFileSync(filePath, req.file.buffer);

    req.savedFile = `/uploads/${filename}`;

    next();
  } catch (error) {
    next(error);
  }
};

module.exports = {
  upload,
  validateAndSaveImage
};
```

------ middleware/validationMiddleware.js ------

```
const { validationResult } = require("express-validator");

const validate = (req, res, next) => {
  const errors = validationResult(req);

  if (!errors.isEmpty()) {
    return res.status(400).json({
      success: false,
      message: "Validation failed",
      errors: errors.array()
    });
```

```
  }


  next();
};


module.exports = validate;
``````
Routes
------ routes/authRoutes.js ------
``````
const express = require("express");
const { body } = require("express-validator");
const rateLimit = require("express-rate-limit");

const {
  register,
  login,
  profile
} = require("../controllers/authController");

const { protect } = require("../middleware/authMiddleware");
const validate = require("../middleware/validationMiddleware");

const router = express.Router();

const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 10,
  message: {
```

```
    success: false,
    message: "Too many authentication attempts. Try again later."
  }
});



// REGISTER
router.post(
  "/register",
  authLimiter,

  [
    body("name")
      .trim()
      .isLength({ min: 2, max: 50 })
      .withMessage("Name must be between 2 and 50 characters"),

    body("email")
      .trim()
      .isEmail()
      .withMessage("Valid email required")
      .normalizeEmail(),

    body("password")
      .isLength({ min: 6 })
      .withMessage("Password must contain at least 6 characters")
  ],

  validate,
  register
```

```
// LOGIN
router.post(
  "/login",
  authLimiter,

  [
    body("email")
      .trim()
      .isEmail()
      .withMessage("Valid email required")
      .normalizeEmail(),

    body("password")
      .notEmpty()
      .withMessage("Password is required")
  ],

  validate,
  login
);

// PROFILE
router.get(
  "/profile",
  protect,
  profile
```

```
module.exports = router;
``````
------ routes/postRoutes.js ------
``````
const express = require("express");
const { body } = require("express-validator");


const {
   createPost,
   getPosts,
   getPostById,
   updatePost,
   deletePost,
   uploadPostImage
} = require("../controllers/postController");


const { protect } = require("../middleware/authMiddleware");


const validate = require("../middleware/validationMiddleware");


const {
   upload,
   validateAndSaveImage
} = require("../middleware/uploadMiddleware");


const router = express.Router();
```

```
router.post(
  "/",
  protect,


  [
    body("title")
      .trim()
      .isLength({ min: 3, max: 150 })
      .withMessage("Title must be between 3 and 150 characters"),


    body("content")
      .trim()
      .isLength({ min: 5 })
      .withMessage("Content is required")
  ],


  validate,
  createPost
);




// GET ALL

router.get(
  "/",
  protect,
  getPosts
);
```

```
// GET ONE

router.get(
   "/:id",
   protect,
   getPostById

);




// UPDATE

router.put(
   "/:id",
   protect,


   [

      body("title")

         .optional()

         .trim()

         .isLength({ min: 3, max: 150 })

         .withMessage("Invalid title"),



      body("content")

         .optional()

         .trim()

         .isLength({ min: 5 })

         .withMessage("Invalid content")

   ],



   validate,
   updatePost
);
```

```
// DELETE

router.delete(
   "/:id",
   protect,
   deletePost
);




// IMAGE

router.post(

   "/:id/image",
   protect,
   upload.single("image"),
   validateAndSaveImage,
   uploadPostImage
);


module.exports = router;
``````

------ routes/viewRoutes.js ------

``````


const express = require("express");


const {

   showLogin,
   showPosts,
   showDashboard
```

```
} = require("../controllers/viewController");

const { protect } = require("../middleware/authMiddleware");

const router = express.Router();

router.get("/login", showLogin);

router.get("/posts", showPosts);

router.get("/dashboard", protect, showDashboard);

module.exports = router;
```

``````

Views

------ views/dashboard.ejs ------

``````

```
<!DOCTYPE html>
<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >
```

```
    <title>Dashboard</title>

    <link rel="stylesheet" href="/style.css">

</head>

<body>

<nav>

    <h2>Dashboard</h2>

    <div>
        <a href="/posts">Public Posts</a>
        <a href="/dashboard">Dashboard</a>
    </div>

</nav>

<div class="container">

    <h1>
        Welcome, <%= user.name %>
    </h1>

    <p>
```

```
    Email: <%= user.email %>
  </p>
  <p>
    Role: <%= user.role %>
  </p>

  <h2>Your Posts</h2>

  <% if (posts.length === 0) { %>
    <div class="card">
      <p>You have not created any posts yet.</p>
    </div>
  <% } %>

  <% posts.forEach(post => { %>
    <div class="card">
      <% if (post.image) { %>
        <img
          src="<%= post.image %>"
          class="featured-image"
```

```
        alt="Featured image"
    >

<% } %>

<h2>
    <%= post.title %>
</h2>

<p>
    <%= post.content %>
</p>

<p>
    Status:
    <% if (post.published) { %>
        <strong>Published</strong>
    <% } else { %>
        <strong>Draft</strong>
```

```
        <% } %>


      </p>
      <div>

        <% post.tags.forEach(tag => { %>


          <span class="tag">
            #<%= tag %>
          </span>


        <% }) %>


      </div>



      <br>



      <button

        onclick="deletePost('<%= post._id %>')"

      >

        Delete

      </button>


    </div>


  <% }) %>
```

```
</div>



<script>
async function deletePost(id) {


  const confirmed =

    confirm("Are you sure you want to delete this post?");


  if (!confirmed) {
    return;
  }


  try {


    const response = await fetch(

      `/api/posts/${id}`,

      {

        method: "DELETE"

      }

    );


    const data = await response.json();


    if (!response.ok) {


      alert(data.message);
```

```
            return;
        }

        alert("Post deleted successfully");

        location.reload();

    } catch (error) {

        alert("Delete failed");

    }
}

</script>

</body>
</html>
``````

------ views/login.ejs ------

``````
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <title>Login - Blog Management</title>

    <link rel="stylesheet" href="/style.css">
</head>

<body>

    <div class="container small">

        <h1>Blog Management</h1>

        <div class="card">

            <h2>Login</h2>

            <form id="loginForm">

                <label>Email</label>

                <input
                    type="email"
                    id="email"
                    required
                >

                <label>Password</label>
```

```
            <input
              type="password"
              id="password"
              required
            >

            <button type="submit">
              Login
            </button>

          </form>

          <p id="message"></p>

        </div>

    </div>

<script>

const form = document.getElementById("loginForm");

form.addEventListener("submit", async (event) => {

    event.preventDefault();

    const email =
        document.getElementById("email").value;
```

```
const password =
  document.getElementById("password").value;

const message =
  document.getElementById("message");

try {

  const response = await fetch(
    "/api/auth/login",
    {
      method: "POST",

      headers: {
        "Content-Type": "application/json"
      },

      body: JSON.stringify({
        email,
        password
      })
    }
  );

  const data = await response.json();

  if (!response.ok) {
    message.textContent =
      data.message || "Login failed";
```

```
            return;
        }

        window.location.href = "/dashboard";

    } catch (error) {

        message.textContent =
            "Something went wrong";

    }

});

</script>

</body>
</html>
``````
------ views/posts.ejs ------
``````
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">
```

```
  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >

  <title>Published Posts</title>

  <link rel="stylesheet" href="/style.css">

</head>

<body>
<nav>

  <h2>My Blog</h2>

  <div>
    <a href="/posts">Posts</a>
    <a href="/login">Login</a>
    <a href="/dashboard">Dashboard</a>
  </div>

</nav>

<div class="container">
```

```
<h1>Published Blog Posts</h1>

<% if (posts.length === 0) { %>

  <div class="card">
    <p>No published posts available.</p>
  </div>

<% } %>

<div class="posts">

  <% posts.forEach(post => { %>
    <div class="card">

      <% if (post.image) { %>

        <img
          src="<%= post.image %>"
          class="featured-image"
          alt="Featured image"
        >

      <% } %>

      <h2>
```

```
    <%= post.title %>
  </h2>

  <p>
    <%= post.content %>
  </p>

  <p>
    <strong>Author:</strong>
    <%= post.author.name %>
  </p>

  <div>
    <% post.tags.forEach(tag => { %>
      <span class="tag">
        #<%= tag %>
      </span>
    <% }) %>
  </div>
</div>
```

```
        <% }) %>

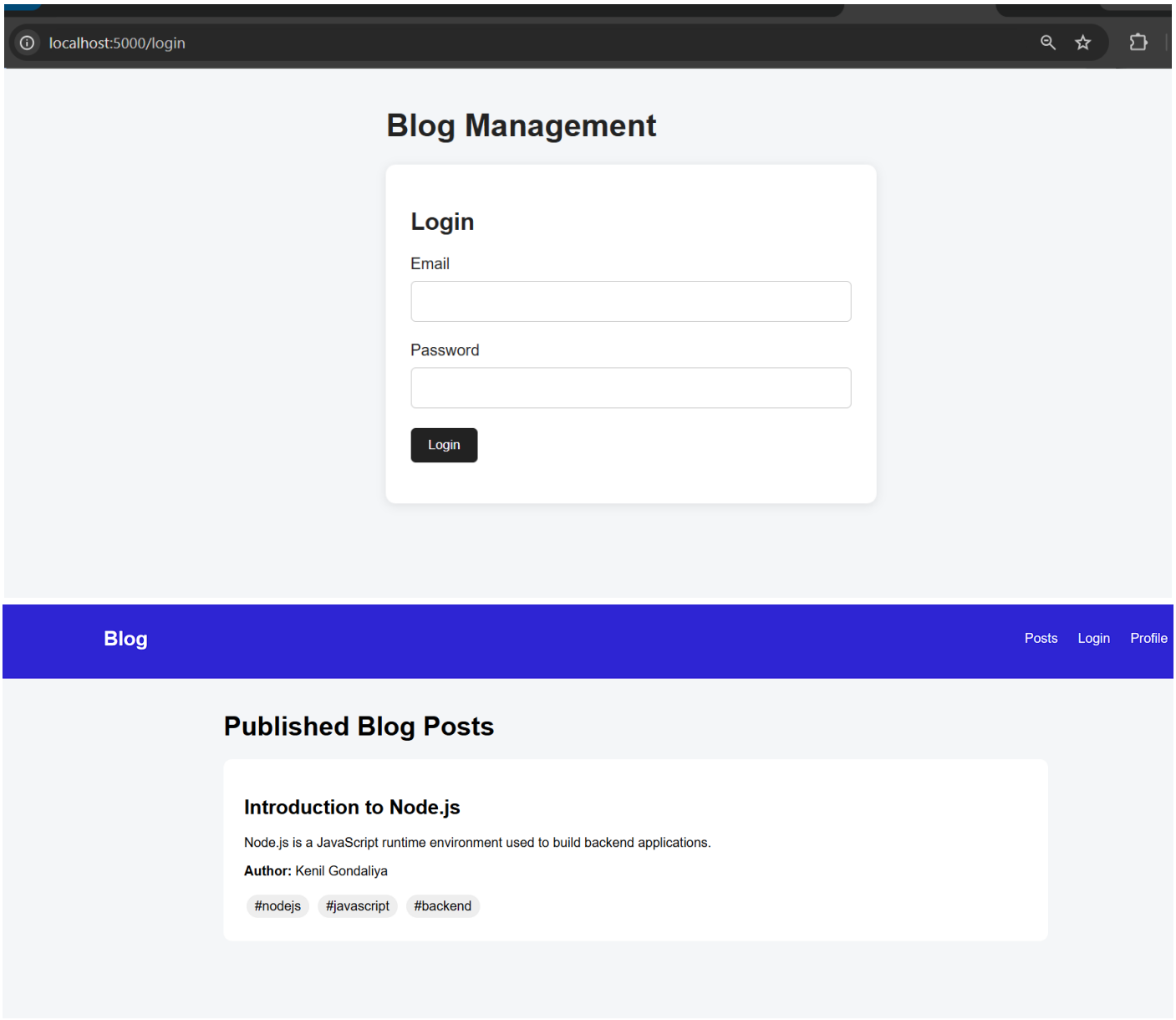

    </div>


</div>


</body>

</html>
```

## Output



# Question 2

Secure Signup for Bookstore Customers

Scenario: A new customer is creating an account on the online bookstore to start placing orders for books.

Task: Write a program that hashes the customer's password using bcrypt before storing it in MongoDB, and write the corresponding logic to verify the password when the same customer logs in later.

Code:

### Server.js

```
const express = require("express");
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");
const Customer = require("./models/Customer");


const app = express();


app.use(express.json());


// Connect to MongoDB
mongoose.connect("mongodb://127.0.0.1:27017/bookstore")
  .then(() => console.log("MongoDB connected"))
  .catch(err => console.log(err));


// SIGNUP
app.post("/signup", async (req, res) => {
  try {
    const { name, email, password } = req.body;


    const existing = await Customer.findOne({ email });
```

```
    if (existing) {
      return res.status(409).json({
        message: "Email already registered"
      });
    }

    // Hash password before saving
    const hashedPassword = await bcrypt.hash(password, 10);

    await Customer.create({
      name,
      email,
      password: hashedPassword
    });

    res.status(201).json({
      message: "Signup successful"
    });

  } catch (err) {
    res.status(500).json({ message: "Signup failed" });
  }
});

// LOGIN
app.post("/login", async (req, res) => {
  try {
    const { email, password } = req.body;

    const customer = await Customer.findOne({ email });
```

```
    if (!customer) {
      return res.status(401).json({
        message: "Invalid email or password"
      });
    }

    // Compare entered password with stored hash
    const isMatch = await bcrypt.compare(
      password,
      customer.password
    );

    if (!isMatch) {
      return res.status(401).json({
        message: "Invalid email or password"
      });
    }

    res.json({
      message: "Login successful",
      customer: {
        id: customer._id,
        name: customer.name,
        email: customer.email
      }
    });

  } catch (err) {
    res.status(500).json({ message: "Login failed" });
```

```
  }

});



app.listen(5000, () => {

  console.log("Server running at http://localhost:5000");

});
```

**customer.js**

```
const mongoose = require("mongoose");



const customerSchema = new mongoose.Schema({
  name: String,
  email: {

    type: String,
    required: true,
    unique: true
  },

  password: {

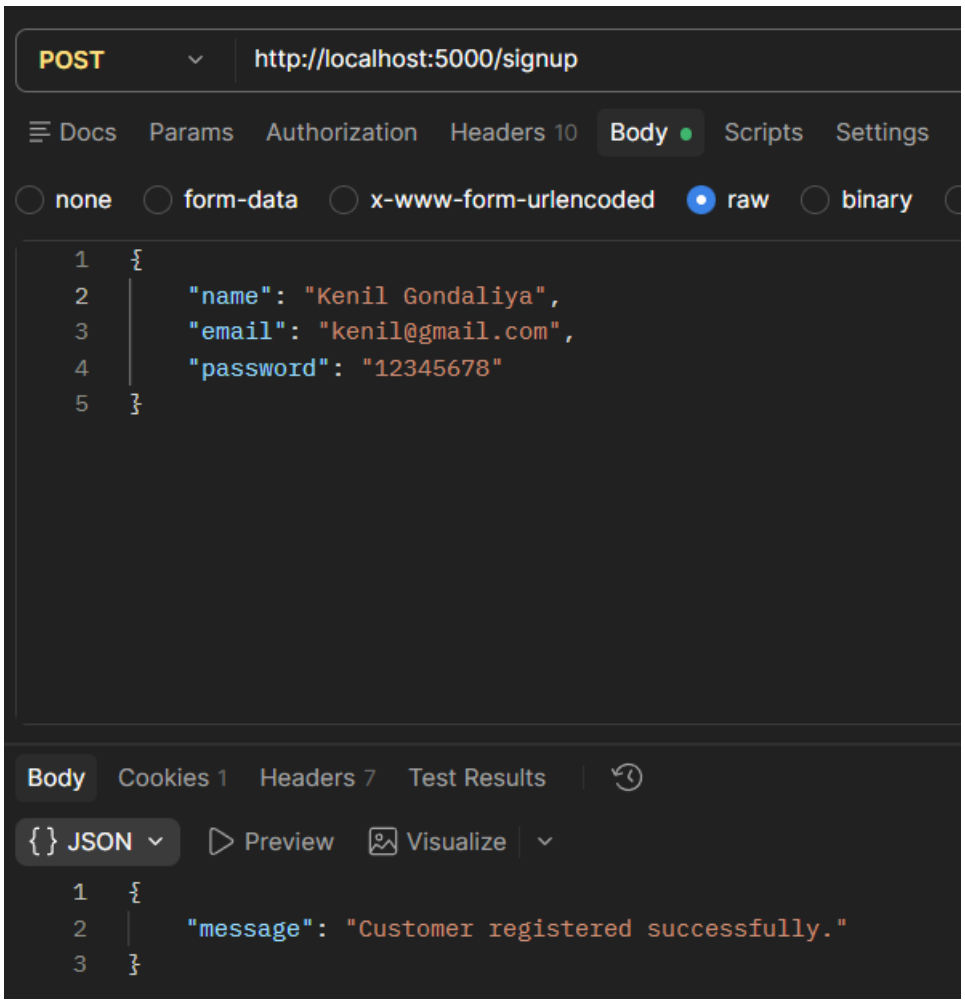
    type: String,
    required: true
  }

});

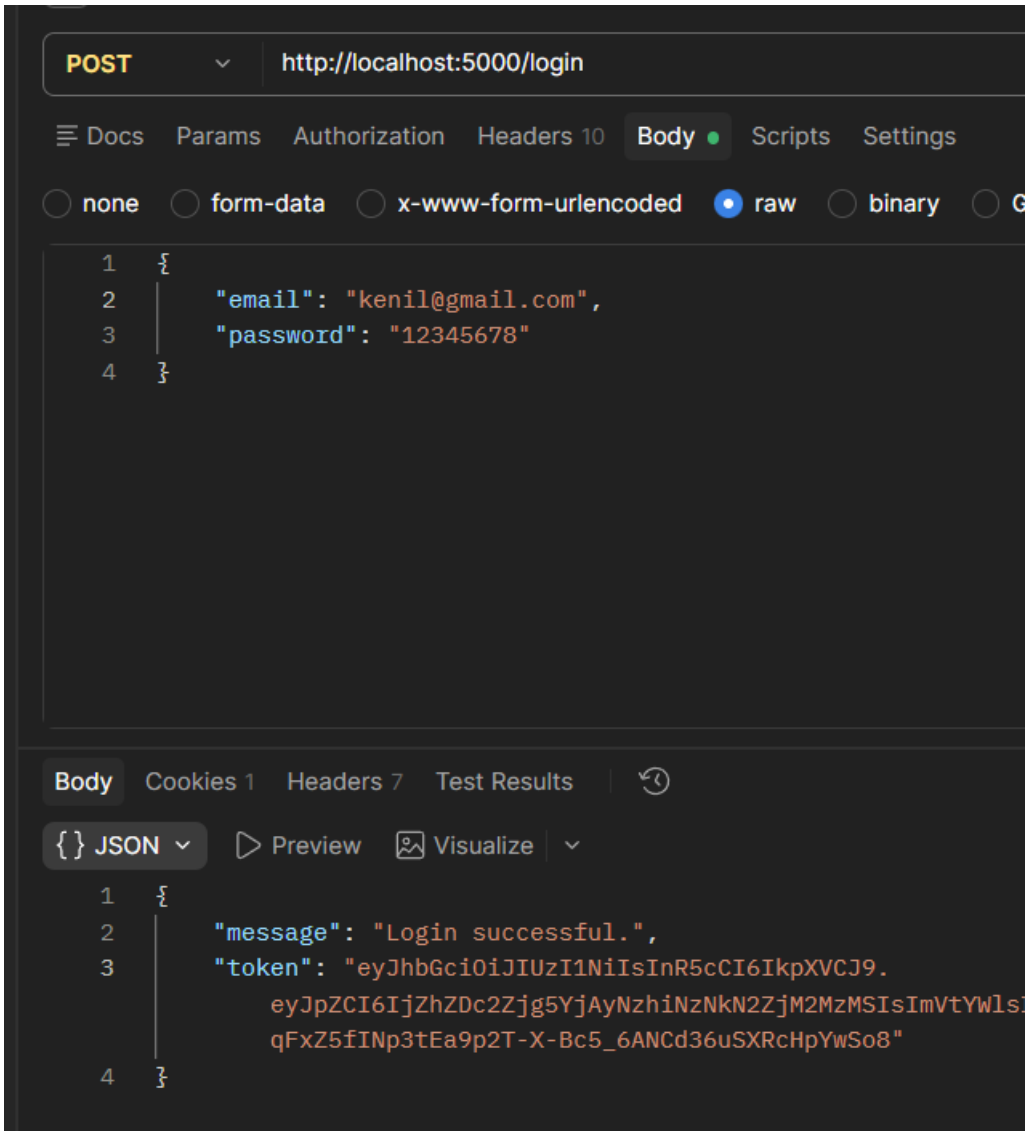


module.exports = mongoose.model("Customer", customerSchema);
```

Output:


POST
http://localhost:5000/signup
Docs
Params
Authorization
Headers 10
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
{
"name": "Kenil Gondaliya",
"email": "kenil@gmail.com",
"password": "12345678"
}
Body
Cookies 1
Headers 7
Test Results
{ } JSON
Preview
Visualize
{
"message": "Customer registered successfully."
}

login Output :



# Question 3

Issuing a Digital Membership Token on Login

Scenario: Once a customer logs in successfully, the store must issue them a token (their digital

membership pass) so they don't need to log in again for every request, such as viewing their orders or checking out.

Task: Write a program to generate a JWT token on successful login, and a middleware to verify this token before allowing access to any protected route (e.g., placing an order or viewing order history).

code

**server.js**

```
require("dotenv").config();



const express = require("express");
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");



const Customer = require("./Customer");

const verifyToken = require("./authMiddleware");



const app = express();
app.use(express.json());


// Connect to MongoDB

mongoose.connect(process.env.MONGO_URI)

  .then(() => console.log("MongoDB connected"))

  .catch(err => console.error("MongoDB error:", err));



// Customer signup

app.post("/signup", async (req, res) => {
  try {
    const { name, email, password } = req.body;
```

```
    if (!name || !email || !password) {
      return res.status(400).json({
        message: "All fields are required."
      });
    }

    const existing = await Customer.findOne({ email });

    if (existing) {
      return res.status(409).json({
        message: "Email already registered."
      });
    }

    const hashedPassword = await bcrypt.hash(password, 10);

    await Customer.create({
      name,
      email,
      password: hashedPassword
    });

    res.status(201).json({
      message: "Customer registered successfully."
    });
  } catch (error) {
    res.status(500).json({ message: "Server error." });
  }
});
```

```
// Customer login and JWT generation
app.post("/login", async (req, res) => {
  try {
    const { email, password } = req.body;

    const customer = await Customer.findOne({ email });

    if (!customer ||
      !(await bcrypt.compare(password, customer.password))) {
      return res.status(401).json({
        message: "Invalid email or password."
      });
    }

    const token = jwt.sign(
      {
        id: customer._id,
        email: customer.email
      },
      process.env.JWT_SECRET,
      { expiresIn: process.env.JWT_EXPIRES_IN || "1h" }
    );

    res.json({
      message: "Login successful.",
      token: token
    });
  } catch (error) {
    res.status(500).json({ message: "Server error." });
  }
});
```

```
// Protected route: view customer orders
app.get("/orders", verifyToken, (req, res) => {
  res.json({
    message: "Access granted. Customer orders can be viewed.",
    customerId: req.customer.id
  });
});



// Protected route: place an order
app.post("/orders", verifyToken, (req, res) => {
  res.json({
    message: "Token verified. Order request accepted.",
    customerId: req.customer.id
  });
});



app.listen(process.env.PORT || 5000, () => {
  console.log("Server running on port 5000");
});
```

**Customer.js**

```
const mongoose = require("mongoose");



const customerSchema = new mongoose.Schema({
  name: String,
  email: {
    type: String,
    unique: true,
    required: true
  },
  password: {
    type: String,
```

```
    required: true
  }
});

module.exports = mongoose.model("Customer", customerSchema);
```

**authMiddleWare.js**

```
const jwt = require("jsonwebtoken");

function verifyToken(req, res, next) {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({
      message: "Access denied. Token required."
    });
  }

  const token = authHeader.split(" ")[1];

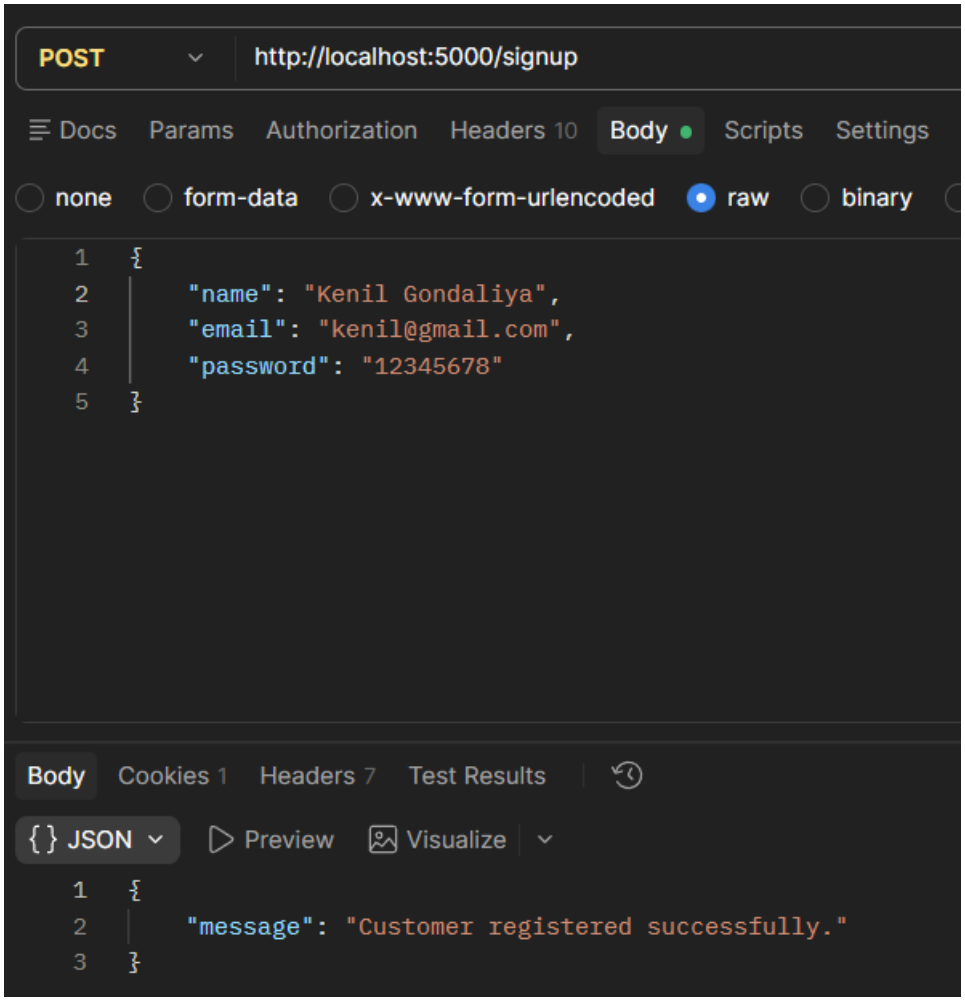
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    req.customer = decoded;
    next();
  } catch (error) {
    return res.status(401).json({
      message: "Invalid or expired token."
    });
```

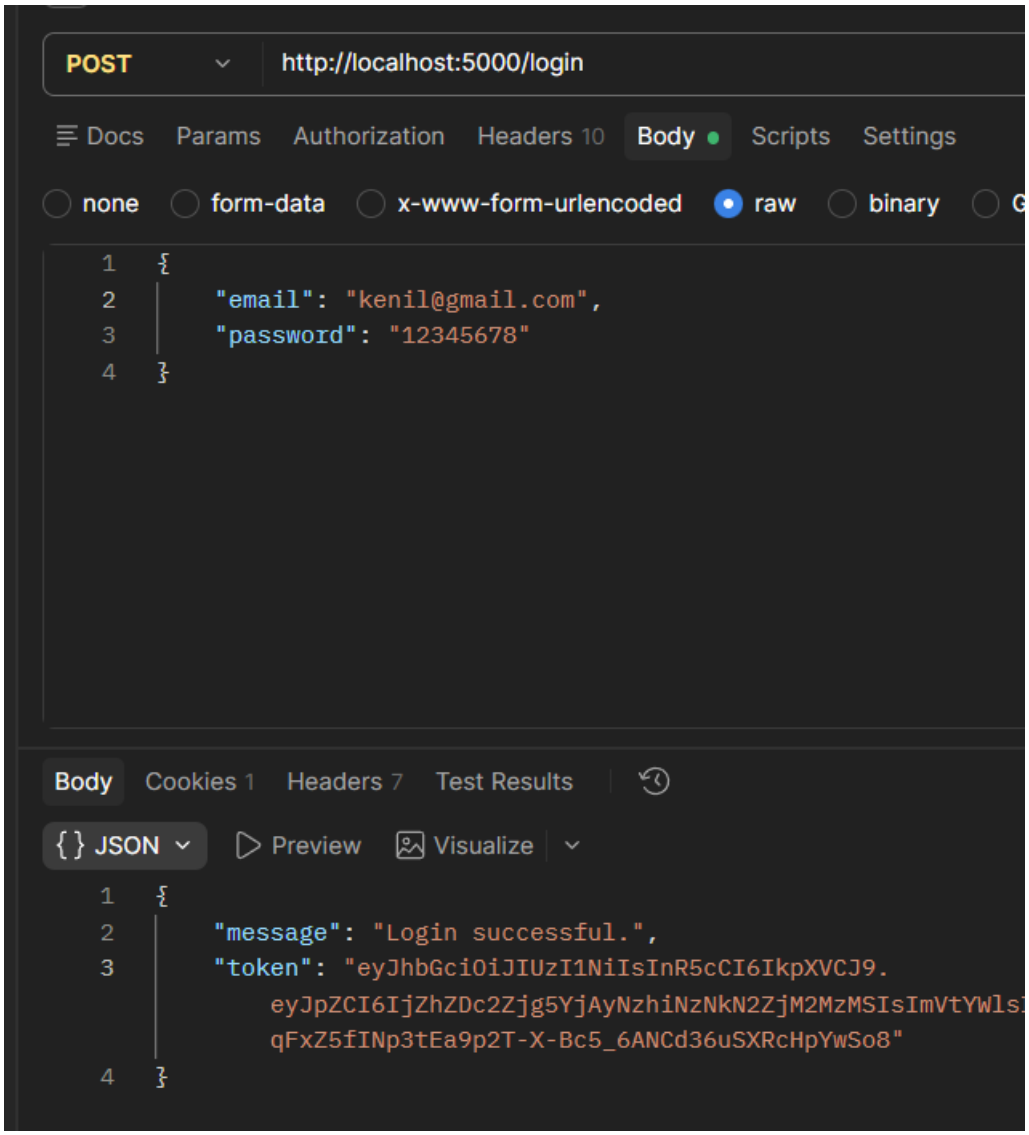
```
  }
}
```

```
module.exports = verifyToken;
```

**output**



login




## Question 4

Defining Customer and Admin Roles

Scenario: The bookstore platform has two kinds of accounts — regular Customers who browse and buy books, and Admins who manage inventory, pricing, and all customer orders.

Task: Write a Mongoose User schema with a role field ('customer' / 'admin') that defaults to 'customer', so every new signup is treated as a regular customer unless explicitly promoted to admin.

code: Server.js

```
const mongoose = require("mongoose");
const User = require("./models/User");


mongoose.connect("mongodb://127.0.0.1:27017/bookstore")

  .then(async () => {
```

```
    const user = await User.create({
      name: "Rahul",
      email: "rahul@example.com",
      password: "hashedPassword123"
    });


    console.log(user);
    process.exit();
  })
  .catch(console.error);
```

**User.js**

```
const mongoose = require("mongoose");


const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },


  email: {
    type: String,
    required: true,
    unique: true
  },

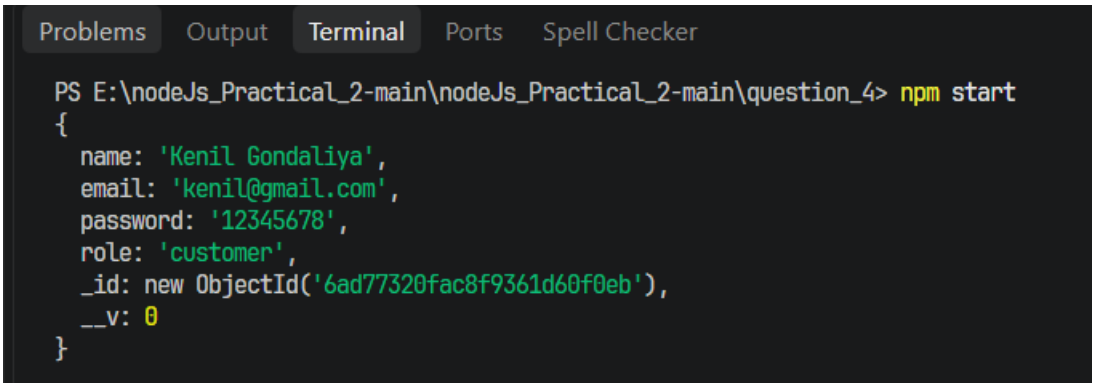

  password: {
    type: String,
    required: true
  },
  role: {
    type: String,
    enum: ["customer", "admin"],
    default: "customer"
```

```
  }

});

module.exports = mongoose.model("User", userSchema);
```

### Output




## Question 5

Admin-Only Access to the Inventory Dashboard

Scenario: The bookstore has an inventory management route (GET /api/admin/inventory) that lists stock levels and sales data for every book, meant only for store admins.

Task: Write an authorization middleware that checks the logged-in user's role and returns 403 Forbidden if the user is not an 'admin', preventing regular customers from accessing this route.

### code

#### authMiddleware.js

```
const jwt = require("jsonwebtoken");



function verifyToken(req, res, next) {

  const authHeader = req.headers.authorization;



  if (!authHeader || !authHeader.startsWith("Bearer ")) {
```

```
      return res.status(401).json({
        message: "Token required."
      });
  }


  const token = authHeader.split(" ")[1];


  try {
    const decoded = jwt.verify(
      token,
      process.env.JWT_SECRET
    );


    req.user = decoded;
    next();
  } catch (error) {
    return res.status(401).json({
      message: "Invalid or expired token."
    });
  }
}


module.exports = verifyToken;
```

**Rolemiddleware.js**

```
function adminOnly(req, res, next) {
  if (req.user.role !== "admin") {
    return res.status(403).json({
      message: "Access denied. Admins only."
    });
```

```
  }


  next();

}


module.exports = adminOnly;
```

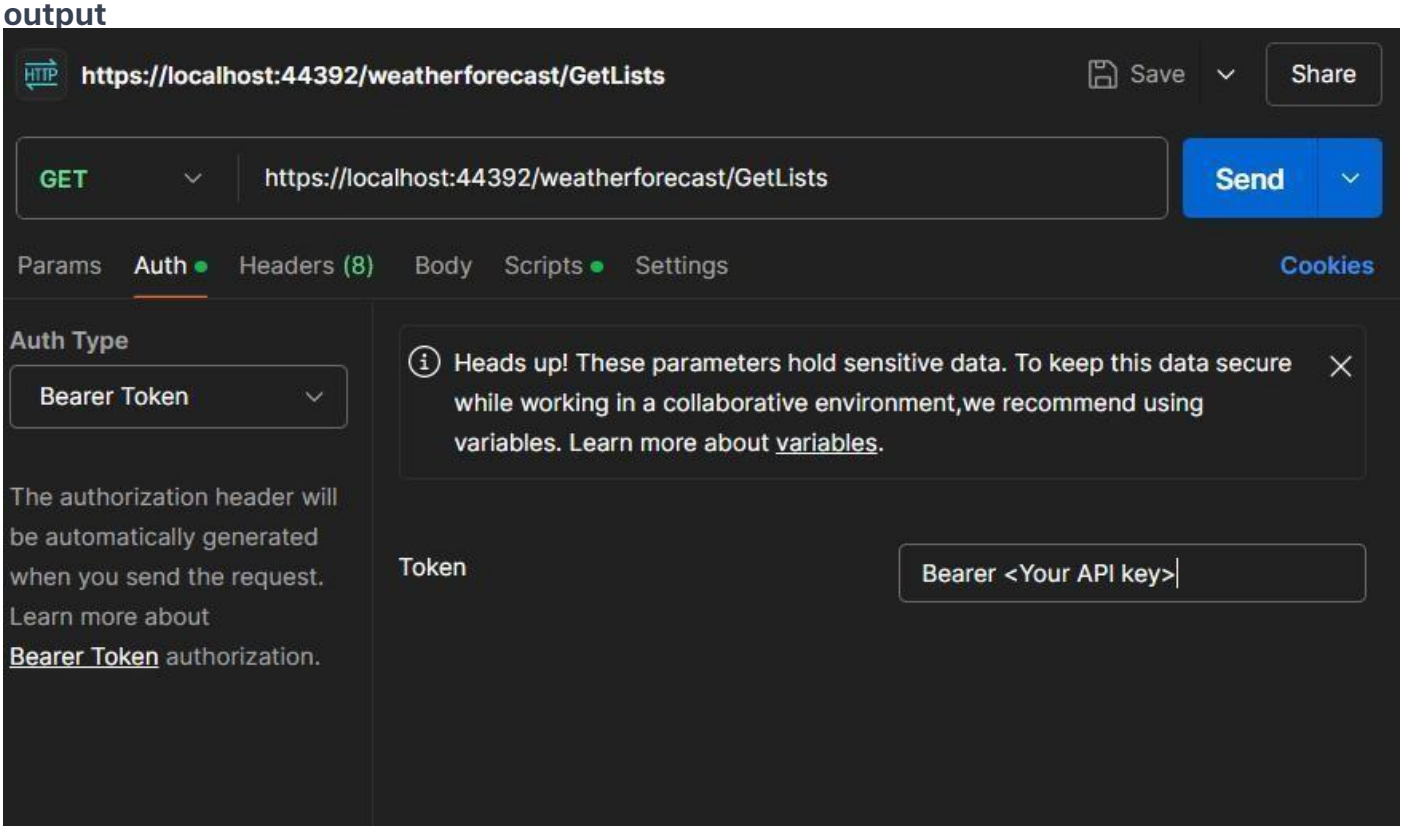
**Customer.js**

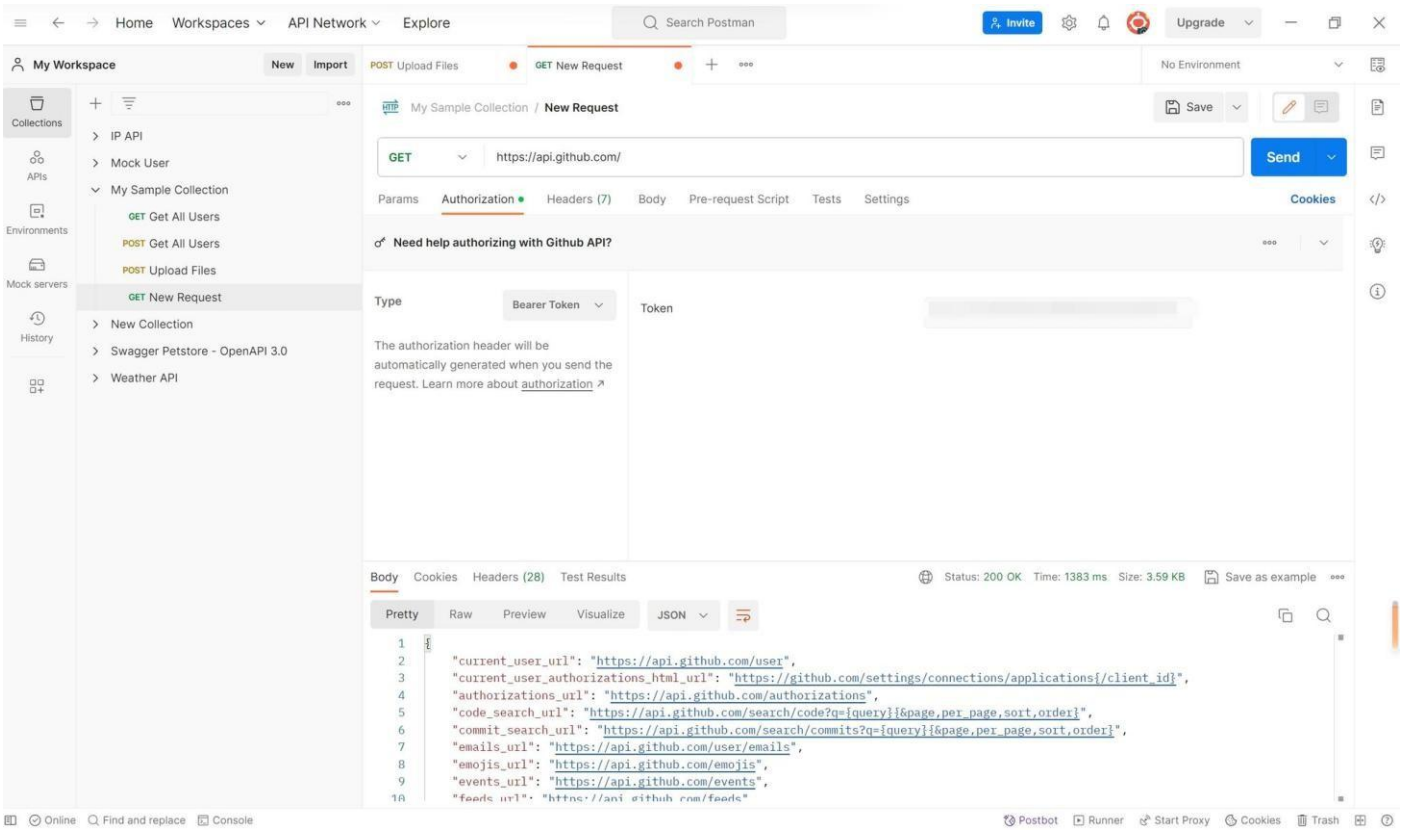
```
const mongoose = require("mongoose");


const customerSchema = new mongoose.Schema({
  name: String,
  email: {
    type: String,
    unique: true,
    required: true
  },
  password: {
    type: String,
    required: true
  }
});


module.exports = mongoose.model("Customer", customerSchema);
```

**output**


https://localhost:44392/weatherforecast/GetLists
Save
Share
GET
https://localhost:44392/weatherforecast/GetLists
Send
Params
Auth
Headers (8)
Body
Scripts
Settings
Cookies
Auth Type
Bearer Token
Heads up! These parameters hold sensitive data. To keep this data secure while working in a collaborative environment,we recommend using variables. Learn more about variables.
The authorization header will be automatically generated when you send the request. Learn more about Bearer Token authorization.
Token
Bearer <Your API key>


Home
Workspaces
API Network
Explore
Search Postman
Invite
Upgrade
My Workspace
New
Import
POST Upload Files
GET New Request
No Environment
Collections
APIs
Environments
Mock servers
History
IP API
Mock User
My Sample Collection
GET Get All Users
POST Get All Users
POST Upload Files
GET New Request
New Collection
Swagger Petstore - OpenAPI 3.0
Weather API
My Sample Collection / New Request
Save
GET
https://api.github.com/
Send
Params
Authorization
Headers (7)
Body
Pre-request Script
Tests
Settings
Cookies
Need help authorizing with Github API?
Type
Bearer Token
Token
The authorization header will be automatically generated when you send the request. Learn more about authorization
Body
Cookies
Headers (28)
Test Results
Status: 200 OK
Time: 1383 ms
Size: 3.59 KB
Save as example
Pretty
Raw
Preview
Visualize
JSON
{
"current_user_url": "https://api.github.com/user",
"current_user_authorizations_html_url": "https://github.com/settings/connections/applications{/client_id}",
"authorizations_url": "https://api.github.com/authorizations",
"code_search_url": "https://api.github.com/search/code?q={query}{&page,per_page,sort,order}",
"commit_search_url": "https://api.github.com/search/commits?q={query}{&page,per_page,sort,order}",
"emails_url": "https://api.github.com/user/emails",
"emojis_url": "https://api.github.com/emojis",
"events_url": "https://api.github.com/events",
Online
Find and replace
Console
Postbot
Runner
Start Proxy
Cookies
Trash

# Question 6

Cancelling an Order — Customer or Admin Only

Scenario: A customer should be able to cancel their own order before it ships. Separately, an admin should be able to cancel any customer's order — for example, when a book goes out of stock.

Task: Write a DELETE /api/orders/:id handler that allows an order to be cancelled only if the logged-in user is the customer who placed the order, or holds the admin role, and returns 403 Forbidden otherwise.

code

**authMiddleware.js**

```
const jwt = require("jsonwebtoken");



function verifyToken(req, res, next) {
  const authHeader = req.headers.authorization;



  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({
      message: "Authentication token required."
    });
  }



  const token = authHeader.split(" ")[1];



  try {
    const decoded = jwt.verify(
      token,
      process.env.JWT_SECRET
    );



    req.user = decoded;
    next();
```

```
  } catch (error) {
    return res.status(401).json({
      message: "Invalid or expired token."
    });
  }
}

module.exports = verifyToken;
```

Order.js

```
const mongoose = require("mongoose");

const orderSchema = new mongoose.Schema({
  customerId: {
    type: mongoose.Schema.Types.ObjectId,
    required: true
  },

  bookName: {
    type: String,
    required: true
  },

  quantity: {
    type: Number,
    default: 1
  },

  status: {
```

```
    type: String,
    enum: ["pending", "shipped", "cancelled"],
    default: "pending"
  }
}, {
  timestamps: true
});

module.exports = mongoose.model("Order", orderSchema);
```

**server.js**

```
require("dotenv").config();

const express = require("express");
const mongoose = require("mongoose");

const Order = require("./models/Order");
const verifyToken = require("./middleware/authMiddleware");

const app = express();
app.use(express.json());

// Connect to MongoDB
mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log("MongoDB connected"))
  .catch(err => console.error("MongoDB error:", err));

// Cancel an order: customer who owns it OR admin
app.delete("/api/orders/:id", verifyToken, async (req, res) => {
```

```
try {

    if (!mongoose.isValidObjectId(req.params.id)) {
      return res.status(400).json({
        message: "Invalid order ID."
      });
    }

    const order = await Order.findById(req.params.id);

    if (!order) {
      return res.status(404).json({
        message: "Order not found."
      });
    }

    const isOwner =
      order.customerId.toString() === req.user.id;

    const isAdmin = req.user.role === "admin";

    if (!isOwner && !isAdmin) {
      return res.status(403).json({
        message: "Forbidden. You cannot cancel this order."
      });
    }

    if (order.status === "shipped") {
      return res.status(400).json({
        message: "Shipped orders cannot be cancelled."
      });
```

```
    }


    if (order.status === "cancelled") {
      return res.status(400).json({
        message: "Order is already cancelled."
      });
    }

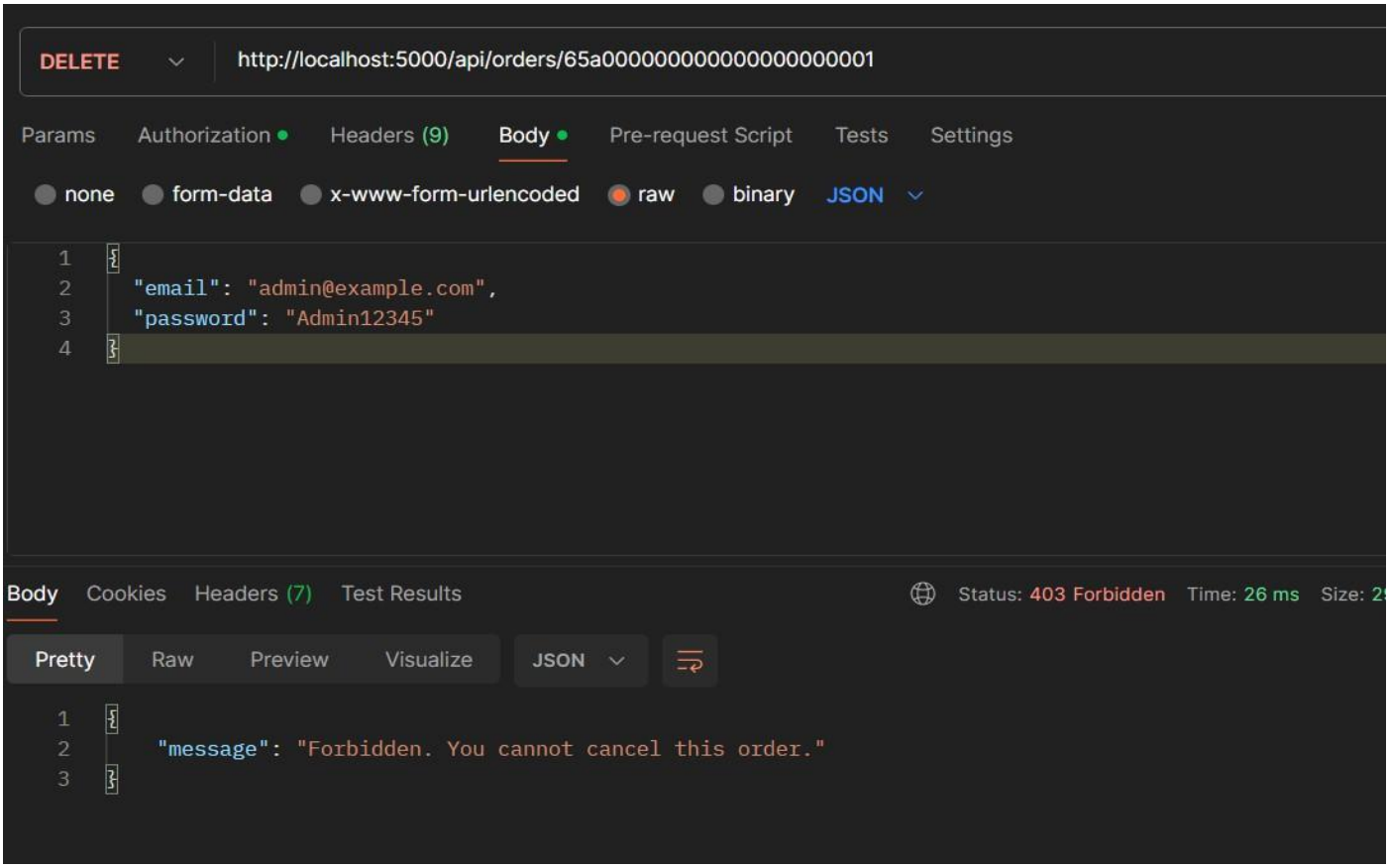

    order.status = "cancelled";
    await order.save();


    return res.json({
      message: "Order cancelled successfully.",
      order: {
        id: order._id,
        bookName: order.bookName,
        status: order.status
      }
    });
  } catch (error) {
    console.error(error);
    return res.status(500).json({
      message: "Internal server error."
    });
  }
});

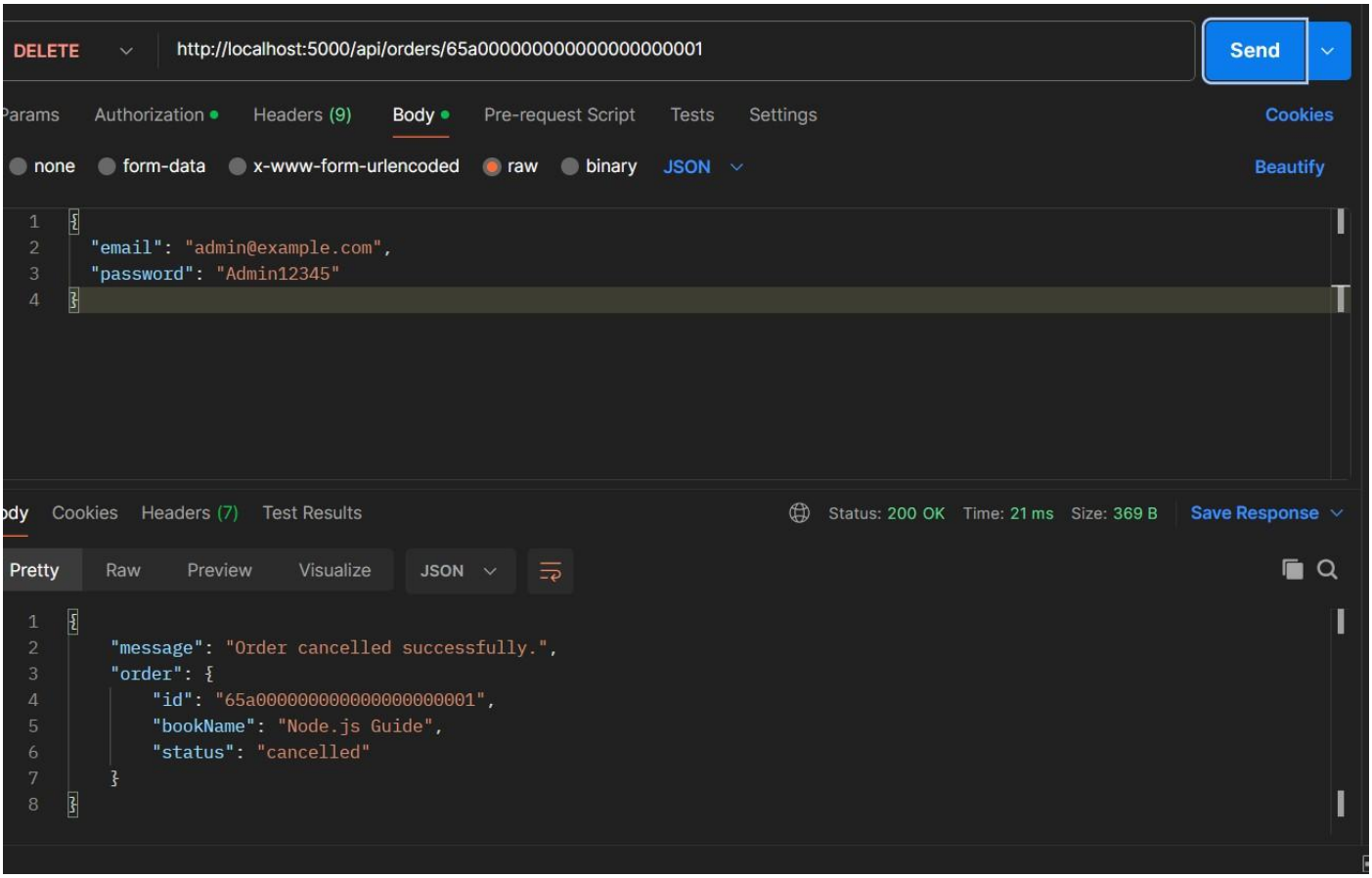

app.listen(process.env.PORT || 5000, () => {
  console.log("Server running on port 5000");
});
```

## Output

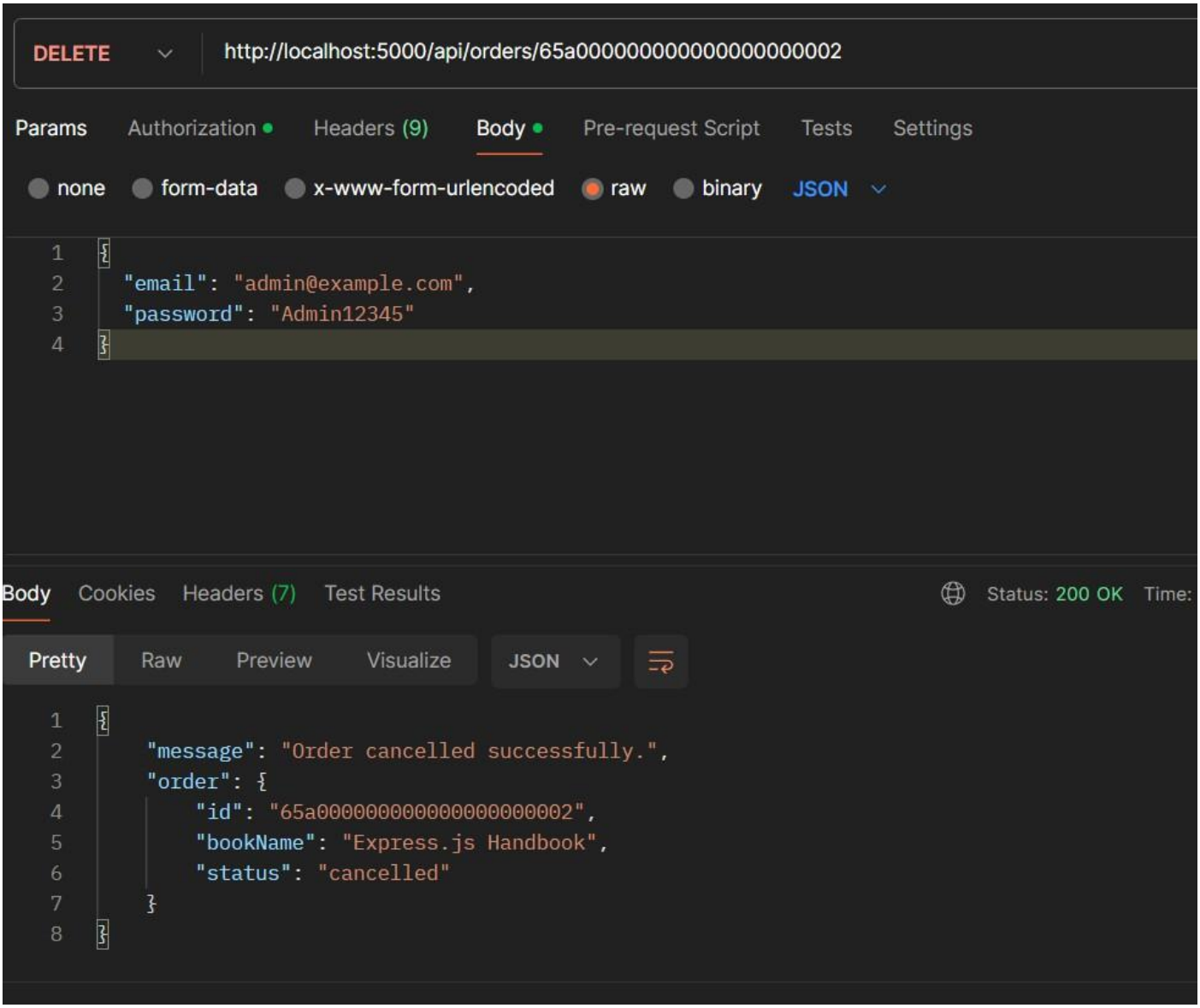
Authorization → Bearer Token → Customer 2 token.




Authorization → Bearer Token → Customer 1 token.


DELETE
http://localhost:5000/api/orders/65a00000000000000000001
Send
Params
Authorization
Headers (9)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
JSON
Beautify
{
"email": "admin@example.com",
"password": "Admin12345"
}
Cookies
Headers (7)
Test Results
Status: 200 OK
Time: 21 ms
Size: 369 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"message": "Order cancelled successfully.",
"order": {
"id": "65a00000000000000000001",
"bookName": "Node.js Guide",
"status": "cancelled"
}
}

Admin token


DELETE
http://localhost:5000/api/orders/65a00000000000000000002
Params
Authorization
Headers (9)
Body
Pre-request Script
Tests
Settings
none
form-data
x-www-form-urlencoded
raw
binary
JSON
{
"email": "admin@example.com",
"password": "Admin12345"
}
Body
Cookies
Headers (7)
Test Results
Status: 200 OK
Time:
Pretty
Raw
Preview
Visualize
JSON
{
"message": "Order cancelled successfully.",
"order": {
"id": "65a00000000000000000002",
"bookName": "Express.js Handbook",
"status": "cancelled"
}
}